

A SUMMER INTERNSHIP TRAINING

at

“GRADTWIN”

Sholinganallur , Chennai

SUMMER TRAINING REPORT

Submitted by

VIGNESH.S - 512722205057

YOGESH.S - 512722205058

YUGASAKTHIVEL.K - 512722205059

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

**SRI BALAJI CHOCKALINGAM ENGINEERING
COLLEGE, ARNI-632 317**

ANNA UNIVERSITY: CHENNAI 600 025

JULY 2025

Certified that this INTERNSHIP REPORT on “ **DATA
SCIENCE** ” is the bonafide work of **YOGESH S
(512722205058)**

who carried out the project work under my supervision.

SIGNATURE

Dr.R. VENKATARATINAM,M.E., Ph.D., HEAD OF THE
DEPARTMENT SUPERVISOR Dept. of Information
Technology,SBC Engineering College, Arni.

SIGNATURE

Mr.S. SELVAM, M.E.. Assistant Professor Dept.of Information
Technology, SBC Engineering College, Arni.

Submitted for the project viva-voice examination held on.....

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any work would be incomplete without mentioning those people who made it possible, whose constant guidance and encouragement rounded our efforts with success.

It is a great pleasure for us to knowledge the assistance and contributions of great people to this effort. First, we would like to thank god for giving us the confidence and power to complete this work successfully.

We would like to thank our founder and chairperson Thiru.Dr.A.C.SHANMUGAM., B.A., B.L., FRCPS(UK) and vice chairperson Thiru.Mr.A.C.S.ARUNKUMAR., B.Tech., M.B.A., and our secretary Thiru.Mr.A.C.RAVI, and Prof.

Dr.V.THIRUNAVUKARASU.,M.E., Ph.D., Principal, Sri Balaji Chockalingam Engineering College, Irumbedu, Arni, who got the whole thing started and who gave us a chance to express ourselves.

We are indebted to acknowledge the effort and substantial contributions of DR. R. VENKATARATHINAM,M.E., Ph.D., Head of the Department, Information Technology, Sri Balaji Chockalingam Engineering College, for giving us the required details, guidelines, valuable comments and suggestions.

We are indebted to acknowledge the efforts and help of our guide Mr.S.SELVAM,M.E., Assistant Professor, Department of Information Technology, Sri Balaji Chockalingam Engineering College, for giving us the required_details,_guidelines.

ABSTRACT

VoiceQuery AI: Intelligent Speech-to-SQL Database Query System, aims to bridge the gap between natural language communication and database querying by enabling users to interact with a structured database using everyday language. Leveraging advanced natural language processing (NLP) techniques and large language models (LLMs) such as GPT, the system converts user voice or text queries into precise SQL commands that retrieve meaningful information from a relational database. The project integrates speech recognition for voice input, a language model for query translation, and an SQLite database for data storage and retrieval, all wrapped in an intuitive web interface built with Streamlit. This approach allows users without SQL expertise to perform complex data retrieval tasks through simple conversational queries, enhancing data-driven decision-making.

TABLE OF CONTENTS

CHAPTER NO NO	TITLE	PAGE
--------------------------	--------------	-------------

ACKNOWLEDGEMENT

ABSTRACT

1	INTRODUCTION	
	1.1 OBJECTIVE	
	1.2 OVERVIEW	
2	PROBLEM STATEMENT	
	2.1 PROBLEM STATEMENT	
	2.2 PROBLEM ANALYSIS	
3	SYSTEM SPECIFICATION	
	3.1 HARDWARE REQUIREMENTS	
	3.2 SOFTWARE REQUIREMENTS	
	3.3 SOFTWARE DESCRIPTION	
	3.3.1 PYTHON	
	3.3.2 VS CODE / JUPYTER	
	NOTEBOOK	
4	SYSTEM DESIGN	
	4.1 SYSTEM ARCHITECTURE	

5

PROJECT WORKFLOW

6

VOICE QUERY AI - INTELLIGENT SPEECH TO SQL DATABASE

4.1 IMPORTING LIBRARIES

4.2 DATASET PREPROCESSING

4.3 DATABASE CONNECTIVITY

4.4 SPEECH RECOGNITION

4.5 QUERY GENERATION AND EXECUTION

CONCLUSION

APPENDICES

APPENDIX 1: SOURCE CODE

APPENDIX 2: SCREEN SHOTS

CHAPTER 1

INTRODUCTION

1.1 OBJECTIVE

The main objective of this project is to develop a voice- based interface that enables users to query a database without the need to write SQL commands manually. By implementing speech recognition technology, the system accurately converts spoken queries into text, which is then parsed and translated into valid SQL statements. The project aims to facilitate seamless interaction with an SQLite database, allowing real-time retrieval of relevant information based on user voice inputs. This approach enhances accessibility by simplifying the querying process for users who may not have technical expertise in SQL, making data retrieval faster and more intuitive.

1.2 OVERVIEW

The Voice Query AI system is designed to bridge the gap between natural language voice commands and database querying. It integrates speech recognition technology with SQL database management to allow users to fetch data simply by speaking their queries. The system captures audio input, converts it to text, processes the text to generate accurate SQL queries, and executes these queries on an SQLite database. This eliminates the need for users to have technical knowledge of SQL, making data access more user-friendly and efficient. The project demonstrates the practical application of AI and natural language processing techniques in simplifying database interactions.

OVERALL STRUCTURE :-

1. Purpose of the System

- To create a voice-based interface that simplifies database querying.
- To allow users to retrieve data without needing SQL knowledge.

2. Key Technologies Used

- Speech recognition via the speech_recognition Python library and Google Speech API.
- SQLite for database storage and management.
- Natural language processing for parsing user queries.

3. System Functionality

- Converts spoken queries into text.
- Translates text into SQL commands.
- Executes SQL queries on the database.
- Displays results to the user in a readable format.

4. Benefits

- Increases accessibility for non-technical users.
- Speeds up the data retrieval process.
- Reduces dependency on database experts.

CHAPTER 2

PROBLEM STATEMENT

2.1 PROBLEM STATEMENT

In many organizations, accessing data from databases requires writing complex SQL queries, which can be challenging for non-technical users. This limitation restricts the ability of users to quickly retrieve information, often leading to delays and dependency on technical staff. The project aims to overcome this barrier by developing a Voice Query AI system that allows users to interact with databases through natural language speech. By converting voice input into SQL queries automatically, the system makes database access more intuitive, faster, and accessible to users without SQL expertise.

2.2 PROBLEM ANALYSIS

Accessing information from databases usually requires writing SQL queries, which can be challenging for users without technical knowledge. This often leads to delays and dependence on technical staff to retrieve data. Even users familiar with SQL can find writing accurate queries difficult and time-consuming, especially for complex data retrieval.

Voice-based interfaces and natural language processing offer a way to simplify this interaction by allowing users to query databases through everyday speech. Developing such a system can significantly improve accessibility and efficiency by enabling non-technical users to get data quickly and intuitively.

By enabling users to interact with databases using natural language voice commands, the system reduces the learning curve associated with SQL and lowers the barrier to data access. This not only speeds up decision-making but also empowers a wider range of users to independently retrieve and analyze data, making organizations more agile and responsive to their information needs.

CHAPTER 3

SYSTEM SPECIFICATION

3.1 HARDWARE REQUIREMENTS

- **Processor:** Intel Core i3 or higher
- **RAM:** Minimum 4 GB RAM
- **Storage:** At least 20 GB free disk space to store datasets and software
- **Microphone:** A good quality microphone or headset for accurate speech input
- **Internet Connection:** Stable internet connection required for using Google Speech-to-Text API
- **Operating System:** Windows 7 or higher / Linux / macOS

3.2 SOFTWARE REQUIREMENTS

- **Programming Language:** Python 3.x
- **IDE/Environment:** Jupyter Notebook / Google Colab / Any Python-compatible IDE
- **Libraries and Packages:**
 - ◆ speech_recognition for converting speech to text
 - ◆ sounddevice and wavio for audio recording and handling
 - ◆ sqlite3 for database connection and management
 - ◆ pandas for data handling and manipulation
- **Database:** SQLite for efficient data storage and querying
- **Operating System Compatibility:** Windows, Linux, macOS

3.3 SOFTWARE DESCRIPTION

3.3.1 PYTHON

Python is a widely used, high-level programming language known for its simplicity and readability. It supports multiple programming paradigms and comes with a vast ecosystem of libraries and tools, making it ideal for data science, machine learning, and AI projects like the Voice Query AI system. Python's flexibility allows easy integration of speech recognition, database management, and audio processing functionalities required in this project

3.3.2 VS CODE / JUPYTER NOTEBOOK

Visual Studio Code (VS Code) is a powerful, lightweight code editor widely used for Python development. It offers rich extensions and debugging tools that streamline coding and testing.

Jupyter Notebook is an interactive coding environment that supports running Python code in cells with immediate output, making it excellent for iterative development and experimentation.

In this project, Jupyter notebooks were run inside VS Code using its built-in support for notebook files. This setup combined the flexibility and interactivity of Jupyter with the powerful editing and project management features of VS Code. It allowed efficient coding, testing, and visualization of the speech recognition and SQL querying components.

CHAPTER 5

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The Voice Query AI system consists of several interconnected components that work together to process user voice input and retrieve relevant data from the database. The architecture follows a modular approach to ensure scalability and maintainability.

Components:

User Interface (Voice Input):

The system captures the user's spoken query through a microphone.

Speech Recognition Module:

Converts the captured audio into text using the speech_recognition library and Google Speech API.

Natural Language Processing & SQL Conversion:

Parses the transcribed text to identify user intent and generates the corresponding SQL query.

Database Module:

Executes the generated SQL query on the SQLite database containing employee data.

Result Display Module:

Retrieves query results and displays them in a user-friendly format

CHAPTER 5

PROJECT WORKFLOW

The workflow of the Voice Query AI system defines the step-by-step process from capturing the user's voice to displaying the final result. This ensures that the system functions efficiently and delivers accurate query outputs.

1. Voice Input Acquisition

The workflow begins when the user interacts with the system by speaking a query into the microphone. The `sounddevice` library in Python captures the live audio stream with a specified sample rate and duration. Once the recording is completed, the `wavio` library stores the captured audio as a `.wav` file, ensuring it is in an uncompressed format suitable for speech recognition processing. This step is crucial because the clarity and quality of the recorded audio directly impact the accuracy of speech-to-text conversion in the later stages.

2. Speech-to-Text Conversion

The saved `.wav` file is processed by the `speech_recognition` library, which acts as the system's Natural Language Processing (NLP) entry point. The audio data is first loaded into memory and then passed to a speech recognition engine, such as Google Web Speech API (default in the library). The engine identifies phonemes and maps them to corresponding words, generating the textual equivalent of the user's spoken query. If speech is unclear or if there is background noise, the recognition process might fail or produce partial results, which can be handled in later error-handling stages.

3. Text Preprocessing

Once raw text is generated, it undergoes preprocessing to make it suitable for query construction. This involves converting all characters to lowercase to maintain uniformity and prevent case-sensitive mismatches in SQL. Extra spaces, punctuation marks, and irrelevant filler words like “please,” “um,” or “can you” are removed to ensure a clean command string. Additionally, the system normalizes terms by mapping synonyms or speech variations to exact column names or table names in the database schema. For example, if the user says “employee salary,” it can be mapped to the salary column of the employee table.

4. SQL Query Construction

In this step, the system interprets the cleaned and normalized text to construct a valid SQL query. A rule-based parsing approach is typically used here, identifying essential components:

Action Keywords: such as “get,” “show,” “list” → mapped to SELECT in SQL.

Target Fields: specific columns requested by the user.

Conditions: constraints like “where salary is greater than 50000” are parsed into SQL WHERE clauses.

5. Database Connection and Execution

The system uses Python's built-in `sqlite3` module to connect to the SQLite database. The connection object opens a communication channel with the database file. A cursor object is then used to send the generated SQL query for execution. The system ensures that the database schema is preloaded or accessible so that SQL statements are valid. This step is critical as any mismatch between the generated query and the schema can lead to execution errors.

6. Result Retrieval and Formatting

After successful execution, the database returns the query results. The results are fetched into Python objects, usually as a list of tuples. To make them human-readable, the data is converted into a structured table format, often using the pandas DataFrame for better presentation. This formatted table includes column headers and organized rows, ensuring clarity when displayed to the user.

7. Output Display

The processed results are displayed in the Jupyter Notebook interface (running inside VS Code). Since Jupyter supports rich text and table rendering, the output is shown as a neat, interactive table. This allows the user to immediately view their requested information in a visually structured form without any need for manual formatting.

CHAPTER 6

VOICE QUERY AI - INTELLIGENT SPEECH TO SQL DATABASE

6.1 IMPORTING LIBRARIES

The first step in the workflow involves importing all the required Python libraries that make the project functional. Each library serves a specific purpose:

wavio (wavio) – Used to save recorded audio in the .wav format. It ensures high-quality audio storage for further processing.

sounddevice (sd) – Enables recording audio directly from the microphone. It captures sound data in real-time for voice input.

speech_recognition (sr) – Converts the recorded audio into text by using speech-to-text processing. It simplifies interaction by allowing natural voice queries.

sqlite3 (sqlite3) – A built-in Python library used to connect and interact with the SQLite database where the dataset is stored.

pandas (pd) – Helps in reading and processing the dataset, enabling data manipulation, cleaning, and querying.

6.2 DATASET PREPROCESSING

Dataset preprocessing involves preparing raw data for effective querying and analysis. This step includes loading the dataset,

handling missing or inconsistent values, converting data types appropriately, and cleaning any noisy data. Preprocessing ensures that the data conforms to the format required by the database and query system, improving accuracy and efficiency during query execution.

```
import pandas as pd

df = pd.read_csv('company_employee_details.csv') # Load dataset

df.isnull().sum() # Check for missing values

df.fillna(df.mean(), inplace=True) # Fill missing values

df.head() # Preview cleaned data

df.tail() # Displays the last five datasets

df.duplicated().sum() # Displays the duplicated datas
```

6.3 DATABASE CONNECTIVITY

Once the dataset has been thoroughly preprocessed—addressing missing values, correcting data types, and cleaning inconsistencies—it is essential to store this clean data in a structured database for efficient querying. SQLite, a lightweight and serverless database engine, is used to manage this data.

Using Python's `sqlite3` library, a connection is established with the SQLite database. The database schema (tables and columns) is defined or created to match the structure of the cleaned dataset. The cleaned data is then loaded into the database tables using pandas DataFrame functionality, which simplifies data insertion.

```
import sqlite3

df = pd.read_csv('company_employee_details.csv') # Load dataset

conn = sqlite3.connect('company_employee.db')

df.to_sql('employees', conn, if_exists='replace', index=False)
```

6.4 SPEECH RECOGNITION

The speech recognition module captures audio input from the user's microphone in real time, converts it into a WAV audio file, and then transcribes the audio into text.

sounddevice is used to record live audio directly from the microphone with a specified duration and sampling rate.

The recorded audio data is saved as a WAV file using **wavio**, ensuring high-quality audio storage for accurate recognition.

The **speech_recognition** library then processes the saved WAV file to convert speech to text using APIs like Google Speech Recognition.

This combination ensures smooth capture and conversion of voice queries into text, which can be further processed for SQL query generation.

6.5 QUERY GENERATION AND EXECUTION

After converting the spoken query to text, the system uses simple if-elif statements to look for keywords in the text. These keywords help decide which SQL query to run.

For example, if the user says “show employees in Sales,” the system looks for the word “Sales” and generates a query to get employees from the Sales department. This method is easy to implement and works well for basic voice commands.

Once the SQL query is created, it is run on the database using sqlite3, and the results are displayed to the user.

```
def convert_to_sql(text):  
    text = text.lower().strip()  
  
    if "employees in" in text:  
        dept = text.split("in")[-1].strip()  
        return f"SELECT * FROM employees WHERE  
                LOWER(TRIM(department)) = '{dept}'"  
  
    elif "age greater than" in text:  
        numbers = [int(s) for s in text.split() if s.isdigit()]  
        return f"SELECT * FROM employees WHERE age >  
                {numbers[0]}" if numbers else None  
  
    elif "salary above" in text:  
        numbers = [int(s) for s in text.split() if s.isdigit()]  
        return f"SELECT * FROM employees WHERE salary >  
                {numbers[0]}" if numbers else None  
  
    return none
```

CONCLUSION

This project successfully demonstrates an intelligent system that converts voice queries into SQL commands to interact with a database. By combining audio recording, speech recognition, and simple query generation techniques, users can retrieve data using natural speech without writing complex SQL.

The use of libraries like `sounddevice`, `wavio`, and `speech_recognition` ensures accurate voice capture and transcription, while `sqlite3` and `pandas` provide efficient data management and query execution.

Although the current implementation uses basic keyword matching for query generation, it lays a strong foundation for future enhancements such as advanced natural language processing and support for more complex queries.

Overall, this Voice Query AI improves accessibility and convenience in database querying, demonstrating practical applications of speech-to-text and database technologies.

APPENDICES

APPENDIX 1 : SOURCE CODE

```
#importing libraries

import sounddevice as sd

import wavio

import speech_recognition as sr

import pandas as pd

import sqlite3


# Data preprocessing - Data Inspection, Data cleaning, Data Exploration

df.shape

df.columns

df.head()

df.tail()

df.info()

df.describe()

df.isnull().sum()

df.duplicated().sum()

df.dtypes


# Connecting to SQL database

import sqlite3

df= pd.read_csv('company_employee_details.csv') # Load dataset
```

```
conn = sqlite3.connect('company_employee.db')  
df.to_sql('employees', conn, if_exists='replace', index=False)
```

```
# Function to record audio and save as .wav
```

```
def record_audio(filename="temp.wav", duration=5, sample_rate=16000):  
    print("Recording... Speak now.")  
    audio = sd.rec(int(duration * sample_rate), samplerate=sample_rate,  
                   channels=1, dtype='int16')  
    sd.wait()  
    wavio.write(filename, audio, sample_rate, sampwidth=2)  
    print("Recording saved.")
```

```
# Function to extract speech from .wav file
```

```
def recognize_speech(filename="temp.wav"):  
    r = sr.Recognizer()  
    with sr.AudioFile(filename) as source:  
        audio = r.record(source)  
    try:  
        text = r.recognize_google(audio)  
        print("You said:", text)  
        return text  
    except sr.UnknownValueError:  
        print("Sorry, could not understand the audio.")
```

```
        return ""

except sr.RequestError as e:

    print(f"Speech Recognition error: {e}")

    return ""
```

Function to convert natural language (text) to SQL query

```
def convert_to_sql(text):

    text = text.lower().strip()

    if "employees in" in text:

        dept = text.split("in")[-1].strip()

        return f"SELECT * FROM employees WHERE  
                LOWER(TRIM(department))='{dept}'"

    elif "age greater than" in text:

        numbers = [int(s) for s in text.split() if s.isdigit()]

        return f"SELECT * FROM employees WHERE age >  
{numbers[0]}" if numbers else None

    elif "salary above" in text:

        numbers = [int(s) for s in text.split() if s.isdigit()]

        return f"SELECT * FROM employees WHERE salary >  
                {numbers[0]}" if numbers else None

    elif "full time" in text:

        return "SELECT * FROM employees  
  
                WHERE full_time = 1"

    elif "part time" in text:

        return "SELECT * FROM employees
```

```

        WHERE part_time = 1"

    elif "contractor" in text:

        return "SELECT * FROM employees

        WHERE contractor = 1"

    return None


# Main part of the program

choice = input("Do you want to speak? (yes/no): ").lower()

if choice == 'yes':

    record_audio()

    query_text = recognize_speech()

else:

    query_text = input("Type your query: ")

# Generate query

sql_query = convert_to_sql(query_text)

print("Generated SQL:", sql_query)

# Execute SQL

if sql_query:

    result = pd.read_sql_query(sql_query, conn)

    print("Query Result:")

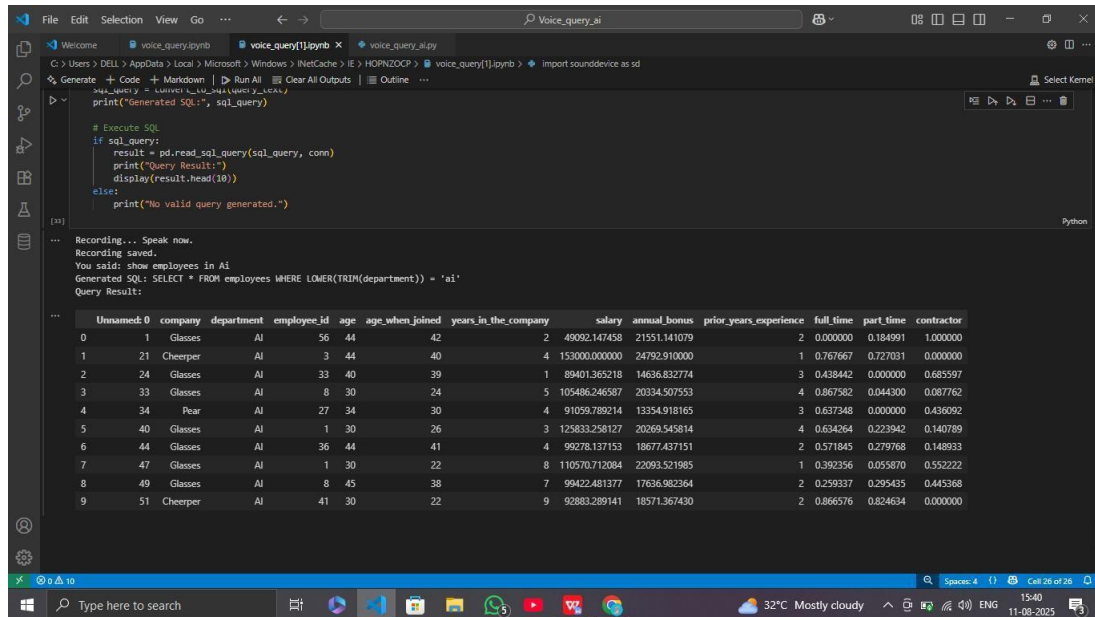
    display(result.head(10))

else:

    print("No valid query generated.")

```

APPENDIX 2 : OUTPUT SCREENSHOTS



```
# Generate SQL query from voice input
sql_query = generate_sql_query(sql_query)
print("Generated SQL:", sql_query)

# Execute SQL
if sql_query:
    result = pd.read_sql_query(sql_query, conn)
    print("Query Result:")
    display(result.head(10))
else:
    print("No valid query generated.")
```

Recording... Speak now.
Recording saved.
You said: show employees in ai
Generated SQL: SELECT * FROM employees WHERE LOWER(TRIM(department)) = 'ai'
Query Result:

	Unnamed: 0	company	department	employee_id	age	age_when_joined	years_in_the_company	salary	annual_bonus	prior_years_experience	full_time	part_time	contractor
0	1	Glasses	AI	56	44	42	2	49092.147458	21551.141079	2	0.000000	0.184991	1.000000
1	21	Cheerper	AI	3	44	40	4	153000.000000	24792.910000	1	0.767667	0.727031	0.000000
2	24	Glasses	AI	33	40	39	1	89401.365218	14636.832774	3	0.438442	0.000000	0.685597
3	33	Glasses	AI	8	30	24	5	105486.246587	20334.507553	4	0.867582	0.044300	0.087762
4	34	Pear	AI	27	34	30	4	91059.789214	13354.918165	3	0.637348	0.000000	0.436092
5	40	Glasses	AI	1	30	26	3	125833.258127	20269.545814	4	0.634264	0.223942	0.140789
6	44	Glasses	AI	36	44	41	4	99278.137153	18677.437151	2	0.571845	0.279768	0.148933
7	47	Glasses	AI	1	30	22	8	110570.712084	22093.521985	1	0.392356	0.055870	0.552222
8	49	Glasses	AI	8	45	38	7	99422.481377	17636.982364	2	0.259337	0.295435	0.445368
9	51	Cheerper	AI	41	30	22	9	92883.289141	18571.367430	2	0.866576	0.824634	0.000000

